Advanced C Projects - Hard Mode

These projects are designed to break reliance on basic if-else ladders and force you to manipulate memory, binary logic, and ASCII values.

Project 5: The IP Subnet Scanner (Bitwise Logic)

In cybersecurity, networks are divided using IP addresses and Subnet Masks. To find the core "Network ID," routers compare the IP and the Mask at the binary level using a Bitwise AND operation.

- **The Task:** Ask the user to input an IPv4 address as four separate integers (e.g., 192, 168, 1, 15). Then ask for a Subnet Mask as four separate integers (e.g., 255, 255, 255, 0).
- **The Math:** Do not use regular addition or multiplication. You must use the C Bitwise AND operator (&) to compare the first IP integer with the first Mask integer, the second with the second, and so on.
- **The Output:** Print the calculated Network ID (which would be 192.168.1.0 in this example).
- **The Trap:** Ensure no input integer goes below 0 or above 255.

Project 6: The Knight's Strike (Grid Mathematics)

You are building the validation engine for a digital chessboard. The user wants to move a Knight, but you must mathematically prove the move is legal.

- **The Task:** Ask the user for the Knight's starting coordinate (X1, Y1) on an 8x8 grid. Then ask for the target coordinate (X2, Y2).

- **The Math:** A Knight moves in an "L" shapou must calculate the absolute difference between the X cooe: exactly 2 squares in one direction and 1 square in the other. Yrdinates and the Y coordinates. (Hint: include <stdlib.h> and use the abs() function, e.g., abs(X1 - X2)).
- **The Filter:** The move is only valid if the X difference is 2 and the Y difference is 1, OR if the X difference is 1 and the Y difference is 2.
- **The Trap:** The target coordinate cannot land outside the 1 to 8 grid boundaries.

Project 7: The Hexadecimal Memory Decoder (ASCII Manipulation)

Memory addresses and colors are often written in Hexadecimal (Base-16), using numbers 0-9 and letters A-F. Your program must convert a 2-character Hex code into a standard decimal integer.

- **The Task:** Ask the user to input two characters (e.g., 'A' and '5', or 'F' and 'F').
- **The Math:** You cannot just print the characters. You must convert them to their actual integer values. If the user types 'A', you must use ASCII math to convert that character into the integer 10. If they type 'C', it becomes 12.
- **The Calculation:** Multiply the integer value of the first character by 16, then add the integer value of the second character. (e.g., 'A' and '5' -> (10 * 16) + 5 = 165).
- **The Trap:** You must handle both uppercase ('A'-'F') and lowercase ('a'-'f') inputs cleanly without writing a massive 24-line if-else maze. Your scanf buffer handling must be flawless to capture two characters consecutively.